

Question 1:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int i;

    for(i = 0; i < 3; i++)
    {
        if(fork() == 0)
        {
            printf("Child %d: PID = %d\n", i + 1, getpid());
            sleep(2);
            printf("Child %d Finished\n", i + 1);
            exit(0);
        }
    }

    for(i = 0; i < 3; i++)
    {
        wait(NULL);
        printf("Parent: One child completed.\n");
    }
}
```

```
        wait(NULL);
        printf("Parent: One child completed.\n");
    }

    printf("All child processes completed.\n");

    return 0;
}
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano program41.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc program41.c -o program41
himasree@LAPTOP-41COT5AA:~/OSLab$ ./program41
Child 1: PID = 709
Child 2: PID = 710
Child 3: PID = 711
Child 2 Finished
Parent: One child completed.
Child 3 Finished
Parent: One child completed.
Child 1 Finished
Parent: One child completed.
All child processes completed.
```

Question 2:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid[3];
    int i;

    for(i = 0; i < 3; i++)
    {
        pid[i] = fork();

        if(pid[i] == 0)
        {
            printf("Child %d: PID = %d\n", i + 1, getpid());
            sleep(2);
            printf("Child %d Finished\n", i + 1);
            exit(0);
        }
    }

    for(i = 0; i < 3; i++)
    {
```

```
        waitpid(pid[i], NULL, 0);
        printf("Parent: Child with PID %d completed.\n", pid[i]);
    }

    printf("All child processes completed.\n");

    return 0;
}
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano program42.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc program42.c -o program42
himasree@LAPTOP-41COT5AA:~/OSLab$ ./program42
Child 1: PID = 719
Child 2: PID = 720
Child 3: PID = 721
Child 1 Finished
Parent: Child with PID 719 completed.
Child 3 Finished
Child 2 Finished
Parent: Child with PID 720 completed.
Parent: Child with PID 721 completed.
All child processes completed.
```

Question 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }
    else if (pid > 0)
    {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());

        printf("Parent sleeping for 15 seconds...\n");
        sleep(15);

        printf("Parent exiting.\n");
    }
    else
```

```
    {
        printf("Fork failed!\n");
    }

    return 0;
}
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano zombie.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc zombie.c -o zombie
himasree@LAPTOP-41COT5AA:~/OSLab$ ./zombie
Parent Process
PID = 728
Parent sleeping for 15 seconds...
Child Process
PID = 729
Child exiting...
Parent exiting.
```